

Flag in Flame

 **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

The SOC team discovered a suspiciously large log file after a recent breach. When they opened it, they found an enormous block of encoded text instead of typical logs. Could there be something hidden within? Your mission is to inspect the resulting file and reveal the real purpose of it. The team is relying on your skills to uncover any concealed information within this unusual log.

Hints

- Use `base64` to decode the data and generate the image file
-

Tools Used

- **file:** Utility to identify file types
 - **strings:** Command-line tool to extract readable ASCII sequences from files
 - **grep:** Text search utility for filtering output
 - **exiftool:** Metadata inspection tool
 - **base64:** Encoding/decoding utility for Base64 data
 - **xxd:** Hexadecimal viewer and editor for converting hex to text
 - **cat:** File reading utility
-

Complete Solution

Step 1: Download and Prepare

Download the log file named `logs.txt` from the challenge page.

logs.txt - Bloc-notes

Fichier Edition Format Affichage Aide

1m63070WACgd8TqdaVmsGN6r6Xu5CL5hd3d9ILz133cuRmG74iXUYrKIhPsWesh0s6js0Iq78irXWet2TteAay8xa1NzaAyGarRrOze6zo8vxeNwcX7nRCIzq9df5d241nQ7v9a9i0FbcNi01pFFHACKSCoAIO
q/Zo0R+wJxk+5rd3ixCRAI1IkFhYIRFGSj12221F0koSrKg0ETstddyDm1XBoDGzrM8mYynxycvty9Yfdrttdv9TKxALd/Zvfe/z4p2Upna7y3mepKub25Nwj+M6dvLUhYgFRacWoPYDnCCD1ts6ytZ1t/PqrL
bw1td+twmqZX19ZaG+nYGGPQIAigapom6Gh65wEAFVtrp9PpFd7v9pVSytrae6+DK0BdTiaTPOT0JpOy1FEUbXe2e72eeCnLsh3lRMRv8vRFvh2N/aomK0GQRd/ib/82zOkieNZfDvVhWHSIvvviidOXdg/du3
Oh1eeh+Fgc/MAAoRq7pmZvEL8DWOY8hNVVh4l6rtTiODw8Pq6ra2nqQZVlRqaZpCfWSJNW0WFFHer0eEY3GF59//vnH5qONjY10rfPy5cvL8bDVamVJBgd9fr8oCrCcJJEknac9mM7bY6XSzmZni0zQrRnZoh
jIwIJo48gPfe0iFAEjBaM2LgxyScZx0zRyQo1NTzjZjItSbHyAcusAEGFPHeem+b4MCI6AGrxtXWsxREXNpzKL1nNmSM0g7o9ZS0yLwwwfshRGQNTBjNgAIYsGok5IFl7Regcq7x3hIBeAB8SQ5Qcy0bEye4K
AyIGUScX0MeIsKqTci7wCG+tvAuBGwdV1YiQMfr8fHDx5AmM5pDnv/07v9vptG6Xqv/zy8clwnz2bjcwaMUKp77ZyTkN/xQdaOmTnlWiIh8rRpbNM0tqmaptnsdP7Vv/pX+7s7BwcH//GvfnJycpIZY4xae09J
lMqLSEw+RAXH8xcPbxdpmhZf5b0PS0072zLWg8kf5F0v3339/oZtrkslk016LHxz4UNhJ80Li2bNne3t7ZeGUUgozImrq2lrb1Fd1lWU7np5PJZDweO+ecxfF4PBqMEDGJKY5jL70maRhs1mVPnz7d3d19+
EAASKTKE6lAzM6AOuT9mFnYIDCi9UzQAbrgSM5IHq79CuAdntq/1FeGAYBEwcLNWsigiHhmRGQW79lXw+IB4iZNI9f2nFW2kSR0LYu46rr00siBeG9Dprhu6jhe6/fX86RtdMyiZtNiN1lomjqkqIwJipbl5
7KyriUBrUVrvZjZ6jdvZRHrLU0wul2bi4t3d3VgnzjVTSUqFoA3EWSRwYFeryIut9cnw5eXls+fp1da7N3bWe92mqabT6ebUtU9GwdBqJc25oB0Xqp/t7d88MOC9n6cBxRSKotJ6P59DjCCiRnmMkZs439joo
iagIBKBpmmqDdV1zVRCrd3Uzn8cxMvNbLgfdO0V9Vw+GwchYAIpdNjHPLpfcem/OyLW/QkSxNnyXtVauQXf/kFsAlcPrJcQc/M3JOac0xHfCO1uWZVmlgTyXZVmZpsV4Xjd1nKYHMBLggy6fHS0Vr4A9ADC8M
e2/ZLwWiZKU8WBDBHqqlkYj04mxnz+d6g3p7N+pILJ0+fpUwLsVJqnWJEVGXtnAugpahbZVnaMcWnW2tdUyZJqoUwKfLiTGv9uw8f3r1799Wtpz/+8Y8vDo/a7XaepwFHSIw1zRNfRwn02k5Vvrrfud2p9PR1
s/F4PG7mFRqTRikiS10LSGooIq7b8MwQESNNRAxv8Myu9+fvPjScCS7NPfc7w9WI+A31/1/7fmLAgdQXf7MM0DOOSTT7XZbvXVjDGMiI1FT2WAwyPxFOIXEZL16KmmMMFprBUGpZa2r6zrP2pubmw9v/89N0
uvqTP/mTo6F4pjje2NiI4xiAnXOerViqinUcx8cHZ1rrPN/I87zb6qVpys6Ezb1SSpgAQCI1NRMwQqr+TJElbKmcCHK8kkiqIbN26cV5VzTnytLq+1Zcnp8Ph8H/92VegdbxsGxsbN1q7/X4/4SgomPw6TZb03
chDQ7kbtdrvT6TCzrTker3Ecs/dVVSHYKIRSBu9Huk7WZYnr86ePn06Go3u3Lmzsbbf6XTGVzyZTLLEJemiTsoiqBorMu3joiHOT087nU4URfv7+0LTwfm517LdbjPVk9EoNtjkinjnXcjdNZoYAMoS5NvNj
D5hFWKSLtclb4looxYQIABBoEQHGFFHM0gWnpp02cfnPvZCaMzbaRmV807Bzb0Dpb0DSbjj8CxsA/EZFN+6qJZ4oQMLgwhcm1AqDLMp90x2WhN5uNc+765mK5viCisswB2Lmo100HDu6evvied+078GwPh/BOL
MJ10B5D7629frzdtvlmeTqeDvY0DTGXZamzPISQp08tkQAYAllrfbAPZGFNT3/y4swLd6vLeIYP1cPdpPqzX/6fQuXXX+v5xbuz03UXGBidBw8aB0gRiNFUDGSwkIgsgmTLwpaR0bsTwL1HdfcfqMeDAa3r
7z8pF1BrLc2OuXfv3pV10RychBQAQDKZzGawL3de1+Uos/z20r1ep3b9Wg0EHhVvSFjrUug51wMrbV2NB09ePDg+YvfnZ2dTfeOkq8v2Xa7d3IPgN7tve2Gefdo3t68KKWU7qqLWltvV21JnHep++SUCCEYz
r0az47BY6X4en4DANrkCGQqeAvvaEfoODkud6dpAg1ARRs4xmiMIiUhtj5EpZQxhfdeqQzReIcxUm5z4UDiRAALcfu3cQhdz3AN6AAQFgnEAwC0qm1MBiHr06yLYsQAAROTdq0jqu6AhgP8/H++FCDo6vqt
GCQ4yvNc6xy9XgvEsi1xne7BZuSen71ZN++TZk4cPHwYKn3/++FLV22GE/+vhCTOfNatm3UCoiCh6ZA7SugaA1pJ5i105jSiIiI2rbq4WwZ112fRw0nKzrOZNqFal+uSEHz16+Pd/PxkN59//v16WY8mU+9ER
y82r7vEQ0lMhNbajeffYrHbFL6Rnu2RznZrNV9ZKZm2aj1LI2995vNpsY43T2FBE5inN0wu0RoIOZj8dHR0dfppHFHixcv1FKzgyO1VCqKiZsr6Ind1z+gcv5uXq/cXSm4jQpN0hPj+Iika2K0iweOKcM6O
kIicC865LD0w0v0U/PeqdxPQVDERGokCcxqVKBiC4jY1t41gX1mVa1VFkHASF12YST8wZndCNCtwcQOegZAEadIeuEUz5yCR5UmVQEKRCTsWQKQcduaeZLng/39zyaTFeealreTmdR1/fbd16tmru0ouJYhH
F80j3az2ayXy1cvXnz89KOnRyc1tj+tg/fuIgoN9sYFs+cYsAiQKQRyWIGwMi0LCXFslMG6VIB1pVf8uuGpjRHTcRMas1pV54nWRuUePPjwkLi4uMCobDcu8TcOXZVmIMZmadAPDDc4FFpBm7KkAe//XP
0xhj4y7bts1syppHEMLly8//fttqqdc73Ug4ikTi6H5U9+8pNUavLVqIdEtDduewDdd+69BjsTwPur1Pe7ziVc+gBTffJoApBjKq6urQXGc3uqWfF3JM25ZazTLjnguaNQCMRqPHjx/X7Te//8Pv//df7X3yy
bnf6AwHATGYOAHjyrbpxZsqj6S/2R8eEtg1tPjxjmb89+911sSAsheVXs1HxadI+Krop995a2zp3cXE+X56G6IuyKEq7Xi8RMVH44CMIEqkQGDsc1KUg/Evj/UEw+jdu8h6A/s79Zes7SYgRbkVzQo+I1jrFV/
Wq1Wg3HUhSF1VapJgRiYQkZivJ72PF25tx/5c7U2rUY3z9KAEAnF0lqtJP01IPL/q3ecuWEqj502Xa36ABO/zfJnWzQ/ldoZwNkqyJx+JE6CyZg6J0DgCLLyrJsqTVisSitefkyfHROTHmn//5n29ubprrrG
1E5D11XIX8K1Q1I1NJ9rOdOcAgAwQMRkdLMHIMDAKVM01mSqwGS5h3RYDB480D8q3d/ev36dVY0j060K1GVVZP3x0MHD+wyEFFZ1iISfQ4AzjX94uo6ITjc5jak8NNK+CXTyikDkt17D4Jaa6eyR0X8IE1xV
3mv+qu2HAK2790MKCa0sBACIgpL5dX5qxLqqNptNXder1QIREZvgNIPsCdFvgkzE96RshcRgghSL2+utdabhk2eU3Lxs0M0acdEAWQBgS5TOCQAKnJ36fXR+s55r3ys4s4v8Be6/9mpk18GZWnkUhsUAUDriti
TG1rLSG1bcu1zjGubuyHbIRsb2/v6PDRcdJ0jbm9ubgbFLKlM81bCPVlMlJCoUACQbQHSvh07IProcZuDiBeh1tRvld03nF7pKpmbiIi+LYR6f8MV+WChgQ+0HVDTK6x3/KtLf0sxaovF9bt3766urrrz4v/vFL
CIF2W8XuFr5K+Of06cJp1IsxL65vTonIWi8iCI6ZWTixhRNBGctJedkHFBKFUWw57mmds598cUXdV0/fJicjUYAcBvaIUlZ1saoEELTNES0v79vJFncXH7z2TffPwVmF9oYY5MuGRCAKKn01CnCCImIokJEr
IqKs8r6VmEjXUCnT+C5sg4HEIMYIyEmi0x2N4/Hw60SbC65teDKZFp77N+3WudEn2aQraUcW90eIO9+X1cBUsR2iUt1eB0MAQDXM0cCRZ0auQ+d2RwAxiRSCkEQBHNzBykbepqvdnb2xupP9vzZayHq0S8u
ra0hnl579flly8pvjicPnj57cnR0dH21EOniaz7UdnhcFg3C0RMYG7vz15eXp6j9vtHD6YHI0Rcr9cJiIQRu93V8Ev1H1Ik5ft3mEzmnfp5WIoUAKArE1laRUT1xo1Go8FgsFqtMjNk95N1WtkeO+fen129f
I6IWRMQsgog0yACgRAMAXAYAnHObWzaz2YTQXnj1zHnvY0yVFP3udP0Bi+UHXtbnQ/1QdMinIpKoy+0YUqPHnQdhEmU1teemXmbmiMiXsSjXDVxkQC9hKNE27tPezp1YDTBUKUUAa2QUQUUUVuDiCyRqNRX
En+9hjvxx/CelyCBs8RTaELmUK2X1nkAMASoTUrRc5CJMw2GY7nzsvoeI2LMxxhhqmsZmuQ94evqp0h20yJfAKs41h4cnde3Xq1N2m0WwFJZ1sVgsXr975RAk7KB3LYKKMYaAwmuSInIKMSKCbHae5QP8FG4fr
/ZkuYmZIQD949g1gtr+//5Nnn+3t7bXrwWkXyG+NMUL27du3r89FAIDOMwAYDofHx8euosFg0LjQ70xs3b4JIpWsrDEvV2EiShRpDKqua2uz3kbvXE7QWdV/2ch+x3z0d7++h/+iofPXBrrj1A6Tb64eHiNiMA

<

Ln 1, Col 1570744

100%

Windows (CRLF)

UTF-8

>

Step 2: Basic File Inspection

First, verify the file type and general information:

```
file logs.txt
```

Expected output:

```
logs.txt: ASCII text, with very long lines (65536), with no line terminators
```

Step 3: Search for Visible Text

The flag may be present as plain text. Try extracting all character strings:

```
strings logs.txt | grep -i "picoCTF"
```

⚠️ If no results appear, proceed to the next step (decoding).

Step 4: Inspect Metadata

Use `exiftool` to examine the file metadata:

```
exiftool logs.txt
```

Complete output:

```
ExifTool Version Number      : 13.50
File Name                    : logs.txt
Directory                   : .
File Size                    : 1592 kB
File Modification Date/Time   : 2025:10:01 01:28:40+02:00
File Access Date/Time        : 2026:05:05 11:38:53+02:00
File Inode Change Date/Time   : 2026:05:05 11:38:49+02:00
File Permissions              : -rw-r--r--
File Type                    : TXT
File Type Extension          : txt
MIME Type                    : text/plain
MIME Encoding                 : us-ascii
Newlines                     : (none)
Line Count                   : 1
Word Count                   : 1
```

Nothing suspicious in the metadata. The file appears to be a single long line of Base64-encoded data.

Step 5: Decode the Base64 Content

Since the file is Base64-encoded, decode it to reveal the hidden content:

```
cat logs.txt | base64 -d > decoded.png
```

This creates a PNG image file named `decoded.png`.

Step 6: Analyze the Decoded Image

Open the decoded image file. You may notice some hexadecimal text within it:



7069636F4354467B666F72656E736963735F616E616C797369735F69735F616D617A696E675F35636363376362307D

7069636F4354467B666F72656E736963735F616E616C797369735F69735F616D617A696E675F6563313938
3466637D

This appears to be hexadecimal encoding.

Step 7: Decode the Hexadecimal String

Convert the hexadecimal string to text using `xxd` :

```
echo  
'7069636F4354467B666F72656E736963735F616E616C797369735F69735F616D617A696E675F656331393  
83466637D' | xxd -r -p
```

Final Result

picoCTF{<REDACTED>}

Key Concepts

- **Base64 Encoding:** A common method to encode binary data as text
- **Hexadecimal Encoding:** Representing binary data using base-16 digits
- **File Analysis:** Examining file types, metadata, and content for hidden information
- **Layered Encoding:** Multiple layers of encoding to conceal data